

Software Technical Specification

# **VISTORA**

## **ESPECIFICAÇÃO TÉCNICA DE SOFTWARE**

SaaS de vistoria imobiliária para criação, execução, evidência fotográfica, aprovação e emissão de laudos digitais.

Responsável técnico: Equipe Vitrine Software

Responsável pelo produto: Vitrine Software

Versão: 1.0

Status: Proposta arquitetural

Rondonópolis

2026

# GUIA DE USO E SUMÁRIO

Este documento define a arquitetura de referência do Vistora. Ele traduz necessidades de negócio em requisitos verificáveis, decisões técnicas e procedimentos operacionais. As métricas propostas devem ser validadas contra o SLA comercial antes da entrada em produção.

## SUMÁRIO

| Seção | Conteúdo                       | Página |
|-------|--------------------------------|--------|
| 1     | Introdução e escopo            | 1      |
| 2     | Requisitos e rastreabilidade   | 2      |
| 3     | Arquitetura e cenários         | 3      |
| 4     | Dados, contratos e integrações | 4      |
| 5     | Qualidade, segurança e testes  | 5      |
| 6     | Implantação e operação         | 6      |
| 7     | Decisões e referências         | 7      |

Convenções. API significa interface de programação; RBAC significa controle de acesso por papéis; RLS significa política de linha no banco de dados; SLO é objetivo mensurável de confiabilidade. Nenhum segredo, token ou dado pessoal deve constar neste documento.

Arquitetura resumida. O Vistora usa frontend React com TypeScript, APIs e workers em C# ASP.NET Core, PostgreSQL como fonte transacional, Redis para cache e coordenação de curta duração e Nginx como ponto de entrada e balanceador. Armazenamento de objetos compatível com S3 é usado para fotos, vídeos e PDFs.

# 1 INTRODUÇÃO E ESCOPO

## 1.1 Identificação e objetivo

| Item         | Definição                                                                                                                  |
|--------------|----------------------------------------------------------------------------------------------------------------------------|
| Sistema      | Vistora - SaaS multiempresa para vistoria imobiliária.                                                                     |
| Problema     | Vistorias em papel, fotos dispersas e laudos inconsistentes atrasam locações, aumentam contestação e dificultam auditoria. |
| Objetivo     | Permitir que uma vistoria seja criada, executada em campo, revisada e emitida com evidências rastreáveis.                  |
| Público      | Administradoras, imobiliárias, vistoriadores, proprietários, inquilinos e equipe de suporte.                               |
| Responsáveis | Produto define regras; Engenharia implementa; Operações responde pela disponibilidade; Segurança revisa controles.         |

## 1.2 Escopo, exclusões e premissas

| Dentro do escopo                                                                                                                  | Fora do escopo                                                                          | Premissas e dependências                                                                            |
|-----------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Cadastro de imóveis, ambientes, checklists, vistorias de entrada e saída, fotos, laudos, assinatura/aceite e trilha de auditoria. | Gestão financeira, anúncio de imóveis, CRM completo e validação jurídica da assinatura. | Navegador moderno, serviço de e-mail, bucket S3 compatível e provedores de identidade configurados. |

## 1.3 Controle de versão

| Versão | Data       | Autor                  | Alteração                            | Aprovação |
|--------|------------|------------------------|--------------------------------------|-----------|
| 1.0    | 16/09/2026 | Engenharia de Software | Especificação inicial da arquitetura | Pendente  |

## 2 REQUISITOS E RASTREABILIDADE

### 2.1 Requisitos funcionais

| ID    | Requisito                                                                              | Prioridade | Critério de aceite                                                        |
|-------|----------------------------------------------------------------------------------------|------------|---------------------------------------------------------------------------|
| RF-01 | Criar imóveis, unidades, ambientes e modelos de checklist por organização.             | Alta       | Um gestor cria modelo e o disponibiliza apenas ao seu tenant.             |
| RF-02 | Executar vistoria em dispositivo móvel, registrar respostas, observações e evidências. | Alta       | Vistoriador salva rascunho e retoma sem perda de itens já persistidos.    |
| RF-03 | Anexar fotos com metadados, associação a item e integridade de arquivo.                | Alta       | A foto fica vinculada a vistoria, ambiente e item; arquivo não é público. |
| RF-04 | Gerar laudo PDF versionado, com evidências e status de aprovação.                      | Alta       | Laudo aprovado é imutável; correção cria uma nova versão.                 |
| RF-05 | Comparar vistoria de entrada e saída por ambiente e item.                              | Média      | Usuário visualiza divergências e evidências lado a lado.                  |

### 2.2 Requisitos não funcionais

| ID     | Atributo        | Meta objetiva                                                  | Evidência                                  |
|--------|-----------------|----------------------------------------------------------------|--------------------------------------------|
| RNF-01 | Desempenho      | p95 de leitura de vistoria abaixo de 400 ms, excluindo upload. | Teste de carga e dashboard.                |
| RNF-02 | Disponibilidade | 99,9% mensal para API pública em produção.                     | SLO calculado por monitoramento.           |
| RNF-03 | Segurança       | Usuário só acessa registros de seu tenant e de seu papel.      | Testes de autorização e RLS.               |
| RNF-04 | Recuperação     | RPO até 15 min e RTO até 4 h para dados transacionais.         | Backup restaurado em exercício trimestral. |

### 2.3 Matriz de rastreabilidade

| Requisito | Design                 | Teste                  | Evidência        |
|-----------|------------------------|------------------------|------------------|
| RF-02     | API Vistorias + PWA    | Integração de rascunho | Pipeline CI      |
| RF-03     | Object storage privado | Contrato de upload     | Log de auditoria |
| RF-04     | Worker de laudo        | E2E de geração         | PDF versionado   |

1. O vistoriador abre um rascunho autorizado. 2. O frontend envia respostas e solicita URL pré-assinada para cada evidência. 3. A API valida tenant, papel e estado da vistoria e grava alterações em transação. 4. Ao concluir, a API cria um job idempotente. 5. O worker gera o PDF, versiona o laudo e registra evento de auditoria. Falhas temporárias sofrem retry limitado; falha final vira alerta operacional.

## 4 DADOS, CONTRATOS E INTEGRAÇÕES

### 4.1 Dados relevantes

| Entidade                    | Classificação        | Retenção                              | Regra principal                                 |
|-----------------------------|----------------------|---------------------------------------|-------------------------------------------------|
| Organization e User         | Pessoal e interno    | Enquanto contrato ativo + prazo legal | tenant_id obrigatório; acesso por RBAC e RLS.   |
| Property, Unit e Inspection | Interno e contratual | Conforme política contratual          | Não misturar dados entre organizações.          |
| Evidence e Report           | Pessoal/contratual   | Conforme contrato e contestação       | Object storage privado; hash e versão no banco. |
| AuditEvent                  | Interno de segurança | Mínimo 24 meses                       | Somente append; não expõe conteúdo sensível.    |

### 4.2 Contrato de API

POST /api/v1/inspections/{inspectionId}/items/{itemId}/evidence

Consumidor: frontend Vistora autenticado.

Entrada: nome, tipo MIME permitido, tamanho e hash opcional. A resposta contém um identificador de evidência e URL pré-assinada de upload, com expiração curta.

Erros: 400 para validação, 401 sem sessão, 403 fora do tenant ou sem papel, 409 para estado incompatível e 429 para limite excedido.

Idempotência: cabeçalho Idempotency-Key em comandos que mudam estado; chave é mantida por 24 h no Redis e o resultado é repetido para reenvios seguros.

### 4.3 Integrações externas

| Sistema               | Protocolo | Autenticação                              | Falha e degradação                                      |
|-----------------------|-----------|-------------------------------------------|---------------------------------------------------------|
| Object storage S3     | HTTPS/S3  | Credenciais de workload; URL pré-assinada | Upload recomeça; metadados só confirmam após validação. |
| E-mail/SMS            | API HTTPS | Secret em cofre                           | Worker tenta novamente; não bloqueia conclusão.         |
| Identidade (opcional) | OIDC      | Client secret/certificado                 | Login indisponível; sessões existentes seguem TTL.      |

### 4.4 LGPD e privacidade

A minimização de dados limita o cadastro ao necessário para a vistoria. Exportação e exclusão devem obedecer prazo contratual e obrigação legal. Logs devem mascarar e-mails, documentos e URLs pré-assinadas. A base legal, papéis de controlador e operador e canal do titular são definidos em instrumentos jurídicos, não neste documento técnico.

## 5 QUALIDADE, SEGURANÇA E TESTES

### 5.1 Controles

| Área            | Controle adotado                                                                                      | Evidência e responsável                                |
|-----------------|-------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Acesso          | OIDC/JWT, RBAC por papel e tenant_id em todas as consultas. RLS como defesa adicional.                | Testes de autorização; Engenharia e Segurança.         |
| Dados           | TLS em trânsito, criptografia gerenciada em repouso, backups protegidos e URLs privadas.              | Configuração de infraestrutura e exercício de restore. |
| Entrada         | Validação de DTOs, allowlist MIME, limite de tamanho, antivírus assíncrono e rate limit no Nginx/API. | Testes de integração e logs de rejeição.               |
| Disponibilidade | Health checks, réplicas da API, timeout, circuit breaker e cache com TTL.                             | Dashboard de SLO e runbook.                            |

### 5.2 Riscos

| Risco                            | Prob./impacto | Mitigação                                               | Gatilho e dono                       |
|----------------------------------|---------------|---------------------------------------------------------|--------------------------------------|
| Upload de evidência indisponível | Média/alto    | Retentativa, rascunho local e monitoramento do storage. | Erro 5xx acima de 2%; Operações.     |
| Vazamento entre tenants          | Baixa/crítico | Filtros obrigatórios, RLS e testes negativos.           | Alerta de acesso anômalo; Segurança. |
| Cache desatualizado              | Média/médio   | TTL curto e invalidação após comando.                   | Divergência de leitura; Engenharia.  |

### 5.3 Plano de testes e qualidade

Unidade: regras de estado, comparação e autorização. Integração: PostgreSQL, Redis, storage simulado e migrações. Contrato: OpenAPI e compatibilidade de respostas. Fim a fim: criação, campo, upload, conclusão e laudo. Carga: leitura, conclusão e geração concorrente. Segurança: SAST, dependências, testes de autorização e revisão de configuração. Toda liberação exige build, testes obrigatórios, revisão de código, migração compatível e plano de rollback.

## 6 IMPLANTAÇÃO E OPERAÇÃO

### 6.1 Ambientes e pipeline

| Item         | Definição operacional                                                                                                                                                            |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ambientes    | Local para desenvolvimento; teste para integração; homologação com configuração semelhante à produção; produção isolada. Dados reais não são copiados para ambientes inferiores. |
| Pipeline     | Build reproduzível -> testes -> análise de segurança -> criação de imagem -> migração compatível -> deploy gradual -> smoke test -> observação.                                  |
| Configuração | Variáveis não secretas por ambiente. Segredos em cofre com identidade de workload e rotação documentada.                                                                         |
| Rollback     | Interromper rollout, voltar imagem anterior e manter migrações aditivas. Alterações destrutivas exigem plano separado e backup validado.                                         |

### 6.2 Observabilidade e suporte

| Sinal                     | O que indica                 | Alerta e primeira ação                                                    |
|---------------------------|------------------------------|---------------------------------------------------------------------------|
| Latência p95 / taxa 5xx   | Degradação da API            | Alerta se SLO ameaçado; verificar Nginx, réplicas, dependências e traces. |
| Fila de jobs e tentativas | Atraso na geração de laudos  | Alertar por idade/falha; inspecionar worker e storage.                    |
| Conexões PostgreSQL       | Saturação ou vazamento       | Alertar por limite; verificar pool e consultas lentas.                    |
| Memória Redis e evictions | Cache ineficiente ou pressão | Revisar TTL, chaves e capacidade; nunca tratar como perda de dados.       |

### 6.3 Procedimentos de operação

O runbook deve conter dashboards, contatos, comando de escala, procedimento de restauração e comunicação de incidente. Logs estruturados devem carregar `correlation_id`, `tenant_id` pseudonimizado, rota e resultado, sem anexos ou dados de autenticação. Alterações de capacidade, SLO, dependência ou procedimento devem atualizar o runbook e esta especificação.

## 7 DECISÕES E REFERÊNCIAS

### 7.1 Registro de decisões arquiteturais

| ADR     | Decisão                                                                        | Alternativas e consequências                                                                |
|---------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| ADR-001 | Adotar monólito modular ASP.NET Core com worker separado.                      | Evita custo inicial de microserviços; exige fronteiras internas claras e contratos de jobs. |
| ADR-002 | PostgreSQL é a fonte de verdade; Redis somente cache e coordenação temporária. | Evita inconsistência causada por cache; invalidação e TTL continuam necessários.            |
| ADR-003 | Fotos e PDFs em object storage privado, não como blobs no banco.               | Escala melhor para mídia; exige política de ciclo de vida e controle de URLs.               |
| ADR-004 | Nginx como ponto de entrada e balanceador L7.                                  | Centraliza TLS, limites e health checks; não substitui autorização na API.                  |

### 7.2 Checklist de aprovação

A aprovação exige que escopo e requisitos sejam compreensíveis sem ler código; fluxos de falha estejam descritos; contratos estejam versionados; autorização e isolamento de tenant tenham teste; backups tenham evidência de restauração; dashboards e runbooks existam; e que não haja segredos, dados de clientes ou URLs privadas no repositório.

## REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 10719:2015: informação e documentação - relatório técnico e/ou científico - apresentação. Rio de Janeiro: ABNT, 2015.

ARC42. Template for software architecture documentation. Disponível em: <https://arc42.org/overview/>.

BROWN, Simon. The C4 model for visualising software architecture. Disponível em:

<https://c4model.com/>. OWASP. Application Security Verification Standard. Disponível em:

<https://owasp.org/www-project-application-security-verification-standard/>.

MICROSOFT. ASP.NET Core documentation. Disponível em: <https://learn.microsoft.com/aspnet/core/>.

POSTGRESQL. Documentation. Disponível em: <https://www.postgresql.org/docs/>.

REDIS. Documentation. Disponível em: <https://redis.io/docs/>.

NGINX. Documentation. Disponível em: <https://docs.nginx.com/>.

